invest — 投研 Agent

Discord 投研助手，部署于远端 AlmaLinux 8 低配 VPS。
单进程架构：Discord Bot + APScheduler 定时调度 + 插件式 skill 模块。
使用 Ansible + Ansible Vault 部署，secrets 加密存储，不落明文。

项目结构

```
invest/
├── README.md
├── .gitignore
├── config.ini.example              # 业务配置模板 → 复制为 config.ini（不入 git）
│
├── agent/                          # Python 应用源码
│   ├── agent.py                    # 主进程：Discord client + APScheduler 调度入口
│   ├── state.py                    # 全局运行状态（agent 维护，传给每次 skill 调用）
│   ├── requirements.txt
│   └── skills/
│       ├── __init__.py             # BaseSkill 基类 + invoke() 统一调用入口
│       ├── report_morning.py       # 早报 skill（CronTrigger，UTC 00:00）
│       ├── report_evening.py       # 晚报 skill（CronTrigger，UTC 12:00）
│       └── market_scan.py          # 市场扫描 skill（IntervalTrigger，每 15 分钟）
│
└── deploy/                         # Ansible 部署
    ├── ansible.cfg                 # Ansible 行为配置（超时、回调格式等）
    ├── inventory.ini               # 目标主机连接信息：host / port / user / key
    ├── deploy_agent.yml            # 主 playbook，调用 common + agent 两个 role
    ├── secrets.yml.example         # Vault 加密模板 → 复制为 secrets.yml 后加密
    └── roles/
        ├── common/                 # 幂等环境初始化，所有主机通用
        │   └── tasks/main.yml      #   VPS 配置检查 / Swap / Python 3.9
        └── agent/                  # agent 专属部署逻辑
            ├── tasks/main.yml      #   目录 / 代码上传 / venv / 依赖 / service
            ├── handlers/main.yml   #   代码或配置变更时自动 restart service
            └── templates/
                ├── env.j2                      # 生成远端 .env（注入 secrets + 业务配置）
                └── invest-agent.service.j2     # systemd unit 模板
```

配置文件职责

三类信息严格分离，每类只有一个填写位置：

文件	负责内容	入 git
`config.ini`	业务配置：app_dir、channel_id、cron、model 等	❌
`deploy/inventory.ini`	连接信息：host、port、user、ssh key 路径	✅ 无 secret
`deploy/secrets.yml`	Secrets：bot_token、gemini_api_key（Vault 加密密文）	✅ 密文

部署时 Ansible 从三处读取配置，经 `env.j2` 模板合并生成远端的 `.env`（权限 600）。

部署流程

前置条件

- 本机已安装 Ansible：`pip install ansible`
- 目标 VPS：AlmaLinux 8，已配置 SSH key 免密登录
- 已在 Discord Developer Portal 创建 Bot，获取 Token、Channel ID、Guild ID

第一步：填写业务配置

```bash
cp config.ini.example config.ini
# 编辑 config.ini，填写：
#   [remote]  app_dir / service
#   [discord] report_channel_id / guild_id
#   [schedule] cron 时间（UTC）
#   [llm]     gemini_model
```

第二步：填写主机连接信息

```bash
# 编辑 deploy/inventory.ini，修改以下字段：
#   ansible_host               = VPS 的 IP 地址
#   ansible_port               = SSH 端口（默认 22）
#   ansible_user               = 登录用户（通常 root）
#   ansible_ssh_private_key_file = 本机 SSH 私钥路径
```

第三步：创建并加密 secrets

```bash
cp deploy/secrets.yml.example deploy/secrets.yml
# 编辑 deploy/secrets.yml，填写：
#   discord_bot_token: "实际 token"
#   gemini_api_key:    "实际 api key"

ansible-vault encrypt deploy/secrets.yml --ask-vault-pass
# 设置一个 Vault 密码，此后每次部署都需要输入
```

第四步：执行部署

```bash
cd deploy
ansible-playbook deploy_agent.yml --ask-vault-pass
```

Ansible 会依次执行：

1. **common role**：收集并打印 VPS 配置概览（CPU / 内存 / 磁盘 / 负载），按需创建 Swap，安装 Python 3.9
2. **agent role**：创建目录，上传代码，创建 venv，安装依赖，生成 `.env`，注册并启动 systemd service

---

日常操作

仅更新代码（跳过环境初始化，最常用）

```bash
ansible-playbook deploy_agent.yml --ask-vault-pass --tags deploy
```

仅查看 VPS 配置（不做任何变更）

```bash
ansible-playbook deploy_agent.yml --ask-vault-pass --tags check
```

更新 secrets 后重新部署

```bash
# 编辑加密文件
ansible-vault edit deploy/secrets.yml --ask-vault-pass

# 重新生成 .env 并重启服务
ansible-playbook deploy_agent.yml --ask-vault-pass --tags secrets
```

查看远端日志

```bash
ssh root@<host> journalctl -u invest-agent -f
```

手动重启 / 停止服务

```bash
ssh root@<host> systemctl restart invest-agent
ssh root@<host> systemctl stop    invest-agent
```

Discord 指令

所有指令响应均为 `ephemeral`（仅触发者可见），不污染日报 channel。

指令	说明
`/ping`	检查 bot 与 Discord 的连接延迟
`/status`	查看 CPU / 内存 / Swap / 磁盘使用率及下次调度时间
`/report morning`	立即触发早报 skill（与定时触发同一代码路径）
`/report evening`	立即触发晚报 skill
`/scan`	立即触发市场扫描 skill
`/reboot`	重启 agent 进程（弹出确认按钮，默认取消，30 秒超时）

调度时间

在 `config.ini` 的 `[schedule]` 段修改，使用 **UTC 时间**，重新部署后生效。

```ini
[schedule]
morning_report_cron          = 0 0 * * *   # UTC 00:00 = 北京 08:00
evening_report_cron          = 0 12 * * *  # UTC 12:00 = 北京 20:00
market_scan_interval_minutes = 15
```

新增 skill

1. 创建 skill 文件

```python
# agent/skills/my_skill.py
from skills import BaseSkill
from state import State
import discord

class MySkill(BaseSkill):
    name = "我的skill"

    async def run(self, channel: discord.TextChannel, state: State) -> None:
        # 失败时直接 raise，BaseSkill.on_error 会自动发错误消息到 channel
        await channel.send("hello from MySkill")
```

2. 在 `agent.py` 挂载

```python
# 实例化
_skill_my = MySkill()

# 挂到调度器（定时触发）
scheduler.add_job(
    lambda: invoke(_skill_my, _get_report_channel(), state),
    IntervalTrigger(hours=1),
    name="我的skill",
    id="my_skill",
)

# 或挂到 slash command（手动触发）
@tree.command(name="myskill", description="触发我的skill")
async def slash_myskill(interaction: discord.Interaction):
    await interaction.response.send_message("执行中...", ephemeral=True)
    await invoke(_skill_my, _get_report_channel(), state)
```

3. 在 Ansible tasks 里添加上传

```yaml
# deploy/roles/agent/tasks/main.yml
# 在"上传 skill 模块"任务的 loop 里加一行:
- my_skill.py
```

4. 重新部署

```bash
ansible-playbook deploy_agent.yml --ask-vault-pass --tags deploy
```

当前状态

所有 skill 处于**测试模式**，生成占位内容，不调用外部 API。
接入 Gemini 时只需替换各 skill 内的 `_generate_content()` / `_scan()` 方法，调度、状态管理、错误处理逻辑不变。